HW08

Sam Ly

March 27, 2026

Chapter 15

1. `print "${drink} will be ready in ${steep + cool} minutes.";`

   What token types would you define to implement a scanner for string interpolation? What sequence of tokens would you emit for the above string literal?

   Rather than defining a `TOKEN_STRING`, I would define a `TOKEN_QUOTE` and a `TOKEN_STRING_VALUE`. This way, we essentially defer some of the complex work to the parser. Then, we also need a `TOKEN_STRING_INTERP_START`

   For the tokens emitted, it would be

   ```
   TOKEN_QUOTE
   TOKEN_STRING_INTERP_START
   TOKEN_IDENTIFIER
   TOKEN_RIGHT_BRACE
   TOKEN_STRING_VALUE
   TOKEN_STRING_INTERP_START
   TOKEN_IDENTIFIER
   TOKEN_ADD
   TOKEN_IDENTIFIER
   TOKEN_RIGHT_BRACE
   TOKEN_STRING_VALUE
   TOKEN_QUOTE
   ```

2. Later versions of C++ are smarter and can handle the above code. Java and C# never had the problem. How do those languages specify and implement this?

   The two approaches to handling this are:

   (a) Context aware scanning/lexing.

   (b) Parser hack.

   Later versions of C++ and C# use a context aware scanner, so it "knows" when it is parsing a type, and will emit the correct token. On the other hand, Java defers this work to the parser. When the parser is currently parsing a generic, it can dynmically "split" the shift operator into multiple generic closing tags.

3. Name a few contextual keywords from other languages, and the context where they are meaningful. What are the pros and cons of having contextual keywords? How would you implement them in your language's front end if you needed to?

   In modern Python, 'match' and 'case' are only keywords if they are used within a match statement. Otherwise, they can be used as variable names. Contextual keywords are useful, but in my opinion

slightly confusing. The reason Python made these keywords contextual is because of backwards compatibility. If these keywords were in from the beginning, I believe they would not be contextual. If I were to implement them, I would choose to let them be a standard identifier, then have the parser check if that identifier should be a keyword.

Chapter 17

1. `(-1 + 2) * 3 - -4`

   Write a trace of how those functions are called. Show the order they are called, which calls which, and the arguments passed to them.

```
compile()
   initScanner("(-1 + 2) * 3 - -4")
   advance()                              // Consumes '('
   expression()
      parsePrecedence(PREC_ASSIGNMENT)
         advance()                        // Consumes '-'
         grouping()                       // Prefix rule for '('
            expression()
               parsePrecedence(PREC_ASSIGNMENT)
                  advance()             // Consumes '1'
                  unary()               // Prefix rule for '-'
                     parsePrecedence(PREC_UNARY)
                        advance()  // Consumes '+'
                        number()   // Prefix rule for '1'
                           emitConstant()
                     emitByte(OP_NEGATE)

                  advance()             // Consumes '2'
                  binary()              // Infix rule for '+'
                     parsePrecedence(PREC_FACTOR)
                        advance()  // Consumes ')'
                        number()   // Prefix rule for '2'
                           emitConstant()
                     emitByte(OP_ADD)

            consume(TOKEN_RIGHT_PAREN)
               advance()               // Consumes '*'

         advance()                       // Consumes '3'
         binary()                        // Infix rule for '*'
            parsePrecedence(PREC_UNARY)
               advance()               // Consumes '-'
               number()                // Prefix rule for '3'
                  emitConstant()
            emitByte(OP_MULTIPLY)

         advance()                       // Consumes '-'
         binary()                        // Infix rule for '-' (subtraction)
            parsePrecedence(PREC_FACTOR)
               advance()               // Consumes '4'
               unary()                 // Prefix rule for '-' (negation)
                  parsePrecedence(PREC_UNARY)
```

```
                    advance()      // Consumes EOF
                    number()       // Prefix rule for '4'
                        emitConstant()
                emitByte(OP_NEGATE)
            emitByte(OP_SUBTRACT)

consume(TOKEN_EOF)
    advance()
endCompiler()
    emitReturn()
        emitByte(OP_RETURN)
```

2. In the full Lox language, what other tokens can be used in both prefix and infix positions? What about in C or in another language of your choice?

   We can also use the (as both an prefix operator for grouping, or an infix operator for initiating function calls.